

# 担当箇所の基本設計・詳細設計

楽譜データ形式・5 パート振り分け・音色データ連携仕様

25G1053 齋藤翔太

2026 年 5 月 18 日

## 本書の目的

本書は、チーム 23「Arduino オーケストラ」における齋藤担当範囲の基本設計・詳細設計をまとめたものである。対象タスクは表 1 の 4 項目である。作成済みの「楽譜データ・発音情報仕様書」で定めた内容をもとに、システム設計書として機能、構成、インターフェース、状態遷移、ハードウェア条件、ソフトウェア構造、処理フロー、および実音解析による音色データとの接続が追える形に整理する。

表 1 本書が対象とする WBS

| WBS | 担当内容                                                                         |
|-----|------------------------------------------------------------------------------|
| 116 | 楽譜データ形式 基本設計. 5 種類の楽器パートを Arduino のメモリに埋め込む方針を決める.                           |
| 128 | 楽譜データ詳細設計. 課題曲を 5 種類の楽器パートに振り分ける.                                            |
| 129 | 楽譜データ詳細設計. テンポ, 音の高さ, 拍をどんな形でデータに書くか決める.                                     |
| 130 | 楽譜データ詳細設計. Arduino から PC へ送る音データの中身, 送信タイミング, Processing が参照する音色データとの対応を決める. |

# 1 基本設計

## 1.1 設計方針

本担当範囲では、5 台の Arduino 楽器ノードが自分のパートの楽譜を保持し、指揮棒用の別マイコンから届く拍情報に合わせて発音情報を PC へ送る仕組みと、実際の楽器音を解析した音色データを Processing で利用する仕組みを設計する。設計方針は次の通りである。

- 楽譜は PC 側ではなく、各楽器ノード (node\_01～node\_05) のプログラムメモリに埋め込む。
- 楽譜はミリ秒ではなく、音高と拍を基準にした相対表現で持つ。
- テンポは楽譜に固定せず、指揮棒用マイコンから届く BPM により実時間へ変換する。
- 通信仕様は塩澤担当の Arduino 設計に合わせ、PC へ送る NOTE は 20 バイト固定長のバイナリ形式に統一する。
- 実音解析で作成した音色 JSON を、instrumentId によって Processing 側から参照できるようにする。
- 5 つの楽器ノードは共通処理を使い、partId, startBeatNo, instrumentId, 楽譜配列だけを差し替える。

## 1.2 機能一覧と FBS 対応

齋藤担当範囲は、チーム全体 FBS のうち F3「楽譜進行」と F4「発音情報配信」に対応する。表 2 に機能一覧を示す。

表 2: 担当範囲の機能一覧

| ID   | 機能            | 役割                                                                     |
|------|---------------|------------------------------------------------------------------------|
| F3.2 | 楽譜データ保持       | 各楽器ノードが自分のパートの楽譜を C 配列として保持する。外部ストレージは使用しない。                           |
| F3.3 | BEAT 同期での音符進行 | 指揮棒用マイコンから届く BEAT 番号とマスタ時刻に合わせ、発火すべき楽譜イベントを選ぶ。                         |
| F3.4 | NOTE イベント生成   | 音符開始時に発音情報を生成する。休符では NOTE を送信せず、消音は Processing 側が durationMs をもとに管理する。 |

| ID   | 機能              | 役割                                                                                            |
|------|-----------------|-----------------------------------------------------------------------------------------------|
| F3.5 | パート固有ロジック       | partId, startBeatNo, instrumentId, 楽譜配列をノードごとに差し替え, トランペット, ホルン, トロンボーン, チューバ, ドラムの5パートを構成する. |
| F4.1 | NOTE パケット生成     | 発音情報を partId, noteNumber, velocity, gate, durationMs, instrumentId へ変換する.                     |
| F4.2 | Processing への送信 | 20 バイト固定長の NOTE を USB Serial で PC へ送信する.                                                      |
| F4.3 | 音色データ対応         | 実音解析で作成した音色 JSON と NOTE 内の instrumentId を対応させ, Processing が楽器ごとの音色で合成できるようにする.                |

### 1.3 システム構成

ユーザは指揮棒用マイコンを振る. 指揮棒用マイコンは BPM と BEAT を 5 台の Arduino 楽器ノードへ送る. 楽器ノードは自分の楽譜を進め, PC 上の Processing へ発音情報を送る. Processing は受け取った NOTE から音を合成し, スピーカへ出力する.

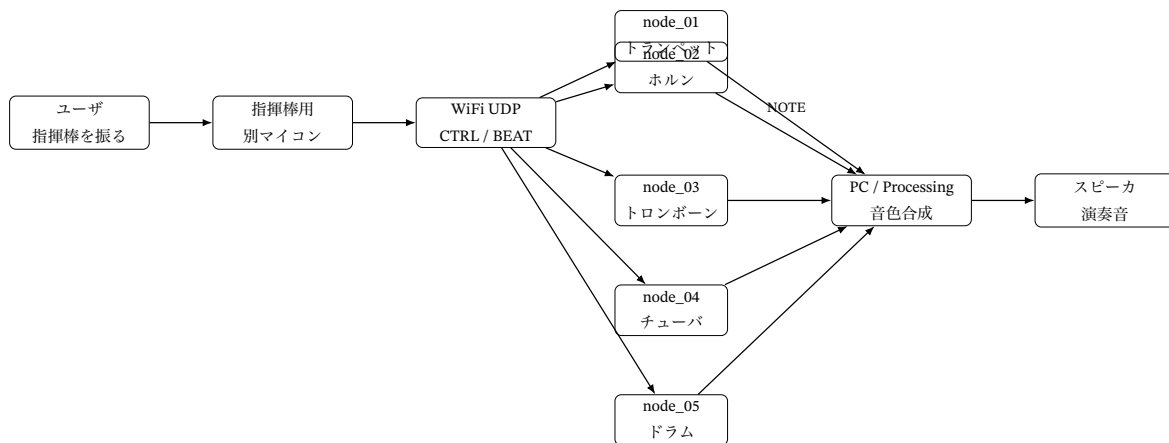


図 1 ユーザ目線のシステム構成

### 1.4 操作インターフェースと状態遷移

齋藤担当範囲では, 専用の物理スイッチや画面は追加しない. ユーザの操作は指揮棒用マイコンを振ることに集約され, 楽器ノードは通信状態と楽譜進行状態に応じて動作する.

表 3 ユーザ操作と楽器ノードの挙動

| ユーザ操作         | 入力される情報             | 楽器ノードの挙動                                |
|---------------|---------------------|-----------------------------------------|
| 電源を入れる        | USB 給電, WiFi 接続開始   | Idle から WaitSync へ進み, CTRL 受信を待つ.       |
| 指揮を振り始める      | BEAT と BPM が届く      | startBeatNo に達したパートから演奏を開始する.           |
| 速く振る／遅く振る     | CTRL の BPM 値が変化する   | 次の音符から新しい BPM で発音長を計算する.                |
| BEAT が一時的に欠ける | BEAT 未受信時間がしきい値を超える | 直前 BPM で SelfRun し, 実 BEAT 復帰後は通常演奏へ戻る. |
| 曲が終わる         | 最終イベントまで発火          | デモ時は曲頭へ戻り, ループ演奏する.                     |

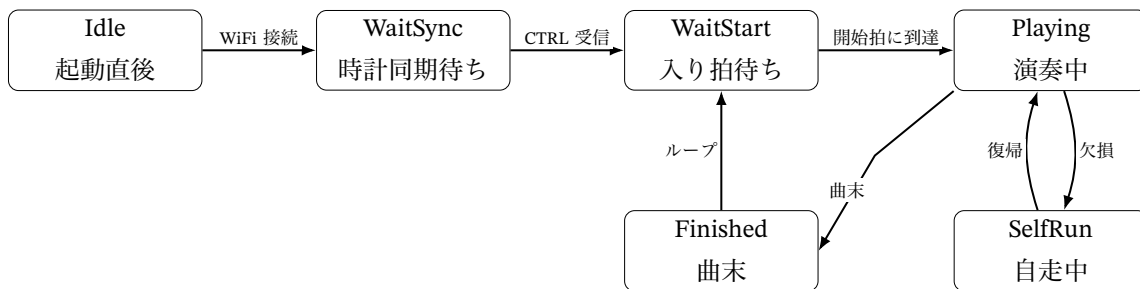


図 2 楽器ノードの状態遷移

## 1.5 5 パート振り分け

課題曲候補は「かえるのうた」とする。輪唱しやすく、テンポ追従の確認がしやすいためである。5 台の Arduino UNO R4 WiFi はすべて楽器ノードとして扱い、指揮棒は別マイコンへ分離する。各 Arduino が担当する楽器は、トランペット、ホルン、トロンボーン、チューバ、ドラムの 5 種類とする。5 パートの割り当てを表 4 に示す。

表 4 5パート振り分け

| ノード     | partId | 役割     | 入り拍 | 楽譜内容                     |
|---------|--------|--------|-----|--------------------------|
| node_01 | 0x01   | トランペット | 0 拍 | 主旋律を曲頭から演奏する。            |
| node_02 | 0x02   | ホルン    | 4 拍 | 主旋律を 4 拍遅れて演奏し、厚みを加える。   |
| node_03 | 0x03   | トロンボーン | 8 拍 | 主旋律を 8 拍遅れて演奏し、輪唱の層を増やす。 |
| node_04 | 0x04   | チューバ   | 0 拍 | 拍の頭を支える低音パートを演奏する。       |
| node_05 | 0x05   | ドラム    | 0 拍 | 1 拍ごとのリズム伴奏を演奏する。        |

## 2 詳細設計

### 2.1 ハードウェア設計

本担当範囲で直接追加する部品はないが、楽譜データと NOTE 送信が前提とする機器、接続関係、動作条件を表 5 と図 3 に示す。

表 5: 使用する機器と役割

| 機器       | 型番・構成                   | 役割                                        |
|----------|-------------------------|-------------------------------------------|
| 楽器ノード    | Arduino UNO R4 WiFi × 5 | node_01～node_05. 楽譜配列を保持し、NOTE を送信する。     |
| 指揮棒用マイコン | 別マイコン × 1               | CTRL / BEAT を WiFi UDP で配信する。楽譜データは保持しない。 |
| PC       | Mac または PC × 1～5        | Processing を実行し、USB Serial から NOTE を受け取る。 |
| 接続ケーブル   | USB Type-C 等            | 楽器ノードと PC を 1 対 1, または USB ハブ経由で接続する。     |
| 音声出力     | PC 内蔵または外部スピーカ          | Processing が合成した音を出力する。                   |
| ストレージ    | 追加なし                    | 楽譜は Arduino のプログラムメモリに埋め込むため、SD カードは使わない。 |

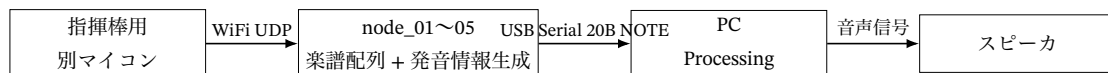


図3 ハードウェア接続関係

## 2.2 ソフトウェア構成

5 台の楽器ノードは共通コードを使い、ProjectConfig.h と score\_data.cpp だけをパートごとに差し替える。これにより、ノード間の処理差分を楽譜データと設定値に限定する。

```

firmware/
|-- node_01/
|   |-- include/
|   |   |-- ProjectConfig.h    // partId, startBeatNo, instrumentId
|   |   '-- score_data.h      // ScoreEvent の宣言
|   '-- src/
|       |-- applyPattern.cpp   // 楽譜進行
|       '-- score_data.cpp     // node_01 用楽譜本体
|-- node_02/
|   |-- include/
|   |   |-- ProjectConfig.h    // partId, startBeatNo, instrumentId
|   |   '-- score_data.h      // ScoreEvent の宣言
|   '-- src/
|       |-- applyPattern.cpp   // 楽譜進行
|       '-- score_data.cpp     // node_02 用楽譜本体
|-- node_03/                  // node_01 と同構造
|-- node_04/
'-- node_05/
  
```

## 2.3 楽譜データ形式

楽譜は ScoreEvent 配列として定義する。小数の float 拍ではなく、durationQ8 という固定小数表現を使う。durationQ8 = 256 を 1 拍とし、テンポ変更時は BPM から実時間へ変換する。

表 6 ScoreEvent のフィールド

| フィールド      | 型        | 意味                                        |
|------------|----------|-------------------------------------------|
| beatAt     | uint16_t | このイベントを発火する拍番号. 各パートの入り拍からの相対値.           |
| noteNumber | uint8_t  | MIDI ノート番号. 60=C4. 0 は休符.                 |
| velocity   | uint8_t  | 楽譜上の強さ. 0~127. CTRL の velocity と合成する.     |
| durationQ8 | uint16_t | 音の長さ. 256 が 1 拍, 128 が 0.5 拍, 512 が 2 拍.  |
| flags      | uint8_t  | bit0=発音イベント, bit2=休符. 将来拡張用の bit は未使用とする. |

```

struct ScoreEvent {
    uint16_t beatAt;        // 発火する拍番号 (パート開始からの相対拍)
    uint8_t noteNumber;    // MIDI ノート番号. 0 なら休符
    uint8_t velocity;      // 楽譜上の強さ 0~127
    uint16_t durationQ8;   // 256 = 1 拍
    uint8_t flags;         // bit0=発音イベント, bit2=休符
};

```

## 2.4 音高・拍・休符の表現

表 7 楽譜データの表現ルール

| 項目    | ルール                                                             |
|-------|-----------------------------------------------------------------|
| 音高    | MIDI ノート番号で表す. C4 は 60, D4 は 62, E4 は 64, F4 は 65, G4 は 67 とする. |
| 休符    | noteNumber = 0 かつ休符フラグを立てる. NOTE は送信せず, 拍だけ進める.                 |
| 拍     | durationQ8 で表す. 四分音符=256, 八分音符=128, 二分音符=512 とする.               |
| テンポ   | 楽譜には BPM を持たせない. CTRL で届く BPM から実時間へ変換する.                       |
| 輪唱の入り | ProjectConfig.h の startBeatNo で指定する.                            |
| 消音    | durationQ8 から durationMs を計算し, Processing 側が発音終了を管理する.          |

## 2.5 パート別設定

```
struct ProjectConfig {
    uint8_t partId;           // 0x01～0x05
    uint16_t startBeatNo;     // 輪唱の入り拍
    uint8_t instrumentId;     // 0=Tp, 1=Hr, 2=Tb, 3=Tu, 4=Dr
    bool loopEnabled;         // デモ時は true
};
```

表 8 ProjectConfig のノード別値

| ノード     | partId | startBeatNo | instrumentId | 楽譜    |
|---------|--------|-------------|--------------|-------|
| node_01 | 0x01   | 0           | 0 トランペット     | 主旋律 A |
| node_02 | 0x02   | 4           | 1 ホルン        | 主旋律 A |
| node_03 | 0x03   | 8           | 2 トロンボーン     | 主旋律 A |
| node_04 | 0x04   | 0           | 3 チューバ       | 低音伴奏  |
| node_05 | 0x05   | 0           | 4 ドラム        | リズム伴奏 |

## 2.6 楽譜データ例

主旋律 A の冒頭は表 9 のように表す。

表 9 主旋律 A 「かえるのうた」冒頭

| 順番 | beatAt | 音名 | noteNumber | durationQ8 | 意味     |
|----|--------|----|------------|------------|--------|
| 1  | 0      | ド  | 60         | 256        | 1 拍鳴らす |
| 2  | 1      | レ  | 62         | 256        | 1 拍鳴らす |
| 3  | 2      | ミ  | 64         | 256        | 1 拍鳴らす |
| 4  | 3      | ファ | 65         | 256        | 1 拍鳴らす |
| 5  | 4      | ミ  | 64         | 256        | 1 拍鳴らす |
| 6  | 5      | レ  | 62         | 256        | 1 拍鳴らす |
| 7  | 6      | ド  | 60         | 256        | 1 拍鳴らす |
| 8  | 7      | 休符 | 0          | 256        | 1 拍待つ  |

```
const ScoreEvent kScore[] PROGMEM = {
    {0, 60, 96, 256, 0x01}, // ド
    {1, 62, 96, 256, 0x01}, // レ
    {2, 64, 96, 256, 0x01}, // ミ
```



```

{3, 65, 96, 256, 0x01}, // ファ
{4, 64, 96, 256, 0x01}, // ミ
{5, 62, 96, 256, 0x01}, // レ
{6, 60, 96, 256, 0x01}, // ド
{7, 0, 0, 256, 0x04} // 休符
};

```

## 2.7 実音解析と音色データ

Processing 側では、単純な電子音だけではなく、各 Arduino が担当する楽器らしい音色で再生できるようにする。そのため、トランペット、ホルン、トロンボーン、チューバ、ドラムの実際の音を WAV 形式で用意し、音声解析によって音色 JSON を作成する。Arduino は波形そのものを送らず、instrumentId と noteNumber を含む NOTE だけを送る。Processing は受け取った instrumentId から音色 JSON を選び、音高、強さ、発音長に合わせて音を合成する。

表 10 音声解析で作成する音色データ

| 項目    | 例                 | 役割                                                 |
|-------|-------------------|----------------------------------------------------|
| 入力音源  | trumpet.wav など    | 実際の楽器音を録音または収集した解析元データ。                            |
| 解析処理  | Python 解析スクリプト    | 基音、倍音成分、音量変化、ノイズ成分を抽出する。                           |
| 出力データ | instrumentId=0 など | Processing が参照する楽器別の音色 JSON。                       |
| 再生処理  | Processing        | NOTE の instrumentId と noteNumber をもとに、対応する音色で合成する。 |

この設計により、Arduino 側は楽譜進行と発音指示に集中でき、音色の複雑な処理は PC 側へ集約できる。また、楽器音を差し替えたい場合も、Arduino の楽譜配列を変更せず、Processing が読む音色 JSON を更新すればよい。

## 2.8 Arduino から PC へ送る NOTE データ

Arduino から PC へ送る NOTE は、20 バイト固定長のバイナリデータとする。共通ヘッダ 12 バイトの後ろに、表 11 の NOTE ペイロード 8 バイトを置く。

表 11 NOTE ペイロード

| offset | field        | type     | 意味                                  |
|--------|--------------|----------|-------------------------------------|
| 12     | partId       | uint8_t  | 0x01~0x05. 発音元パート.                  |
| 13     | noteNumber   | uint8_t  | MIDI ノート番号. 休符は送らない.                |
| 14     | velocity     | uint8_t  | 0~127. 楽譜値と CTRL 値の合成後.             |
| 15     | gate         | uint8_t  | 1=発音開始. 0 は将来の強制消音用に予約する.           |
| 16     | durationMs   | uint16_t | 発音予定長さ. Processing 側が消音時刻を決めるために使う. |
| 18     | instrumentId | uint8_t  | Processing が参照する音色 JSON の ID.       |
| 19     | 予約           | uint8_t  | 0 で埋める.                             |

音符イベント発火時は gate=1 の発音情報を送る. 消音は Arduino から別 NOTE を送るのではなく, Processing 側が durationMs に基づいて行う. gate=0 は, 将来, 途中停止や強制消音が必要になった場合の拡張用として残す. 休符イベントでは NOTE を送らず, 楽譜ポインタだけを進める.

## 2.9 内部 API

表 12: 楽譜進行で使用する内部 API

| 関数                              | 役割                                         |
|---------------------------------|--------------------------------------------|
| loadScoreEvent(index)           | PROGMEM 上の kScore[index] を RAM 上の一時変数へ読む.  |
| isRest(event)                   | noteNumber == 0 または休符フラグから休符か判定する.         |
| durationQ8ToMs(durationQ8, bpm) | durationQ8 を現在 BPM に基づきミリ秒へ変換する.           |
| mergeVelocity(score, ctrl)      | 楽譜上の velocity と指揮者側 CTRL の velocity を合成する. |
| emitNoteEvent(event)            | 発音対象の ScoreEvent から NOTE 送信モジュールへ渡すデータを作る. |
| getInstrumentId()               | ProjectConfig で指定した音色 ID を NOTE ペイロードへ入れる. |
| advanceScore(beatNo)            | BEAT 番号から発火対象イベントを順に処理する.                  |
| buildNotePacket()               | 共通ヘッダと NOTE ペイロードを 20 バイト列にシリアルライズする.      |

## 2.10 処理フロー

楽器ノードは入力、ロジック、出力の3フェーズで動作する。楽譜データに関する処理は `advanceScore()` に集約する。

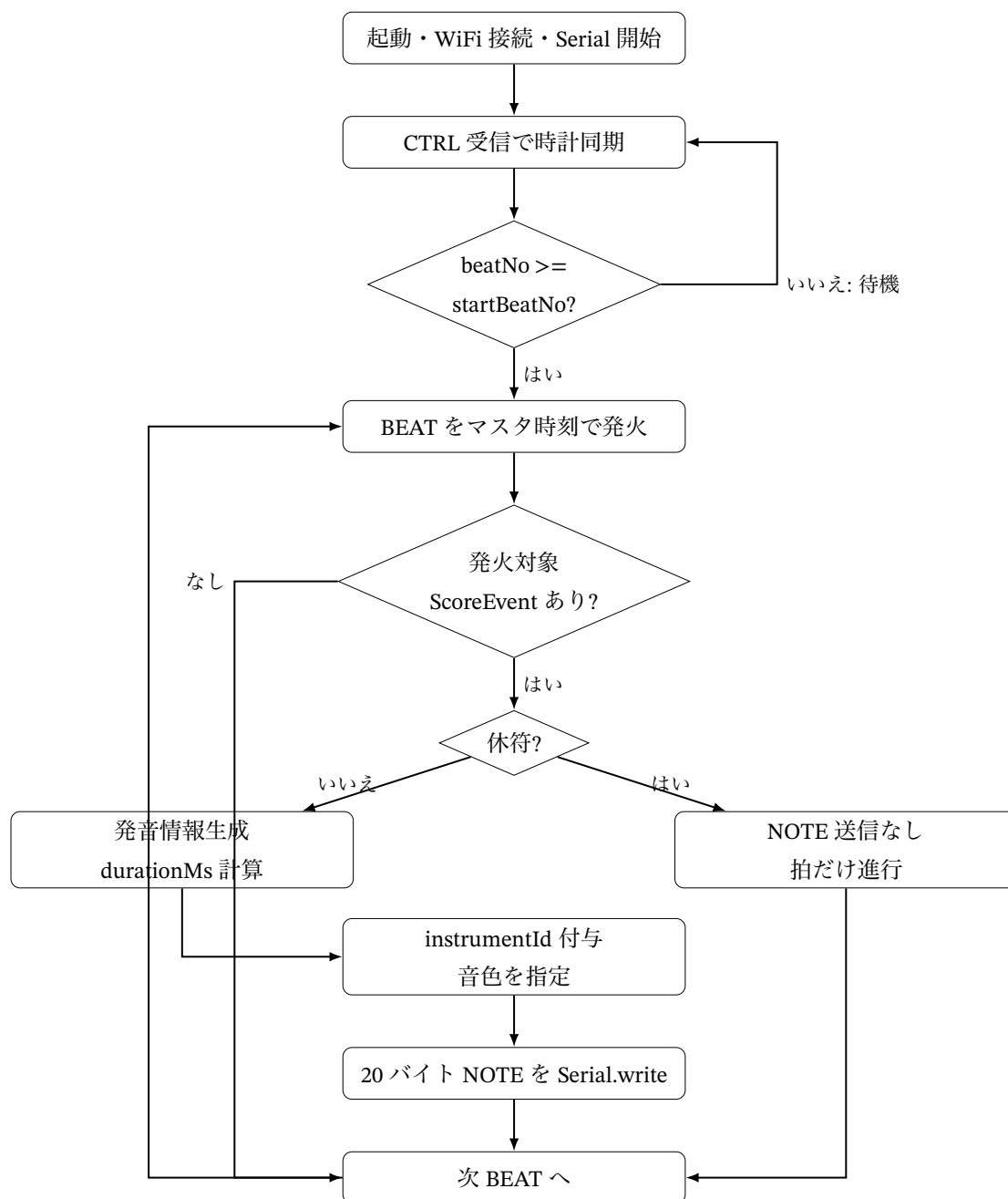


図4 楽譜進行と NOTE 送信の処理フロー

## 2.11 BEAT 受信時の処理

1. 受信した BEAT の beatNo と playAtMasterMs をキューに入れる.
2. beatNo < startBeatNo の場合は, 自パートの入り前なので演奏しない.
3. effectiveBeatNo = beatNo - startBeatNo を計算する.
4. kScore[currentEventIndex].beatAt <= effectiveBeatNo のイベントを順に読む.
5. 休符なら NOTE を送らず, currentEventIndex だけ進める.
6. 音符なら現在 BPM から発音長を求め, instrumentId を含む NOTE を送る.
7. Processing 側は durationMs をもとに発音終了時刻を決め, 音を止める.

## 2.12 テンポ変更時の処理

楽譜データは拍単位で記録するため, テンポ変更時に配列を書き換える必要はない. CTRL で新しい BPM を受け取った後, 次に発火する音符から式 1 で実時間を計算する.

$$durationMs = \frac{60000}{BPM} \times \frac{durationQ8}{256} \quad (1)$$

すでに鳴っている音については, 発音開始時に Processing へ渡した durationMs を優先する. これにより, 途中で BPM が変わっても鳴りかけの音が不自然に切れない.

## 2.13 例外処理

表 13: 例外・分岐時の処理

| 状況          | 判定条件                              | 処理                               |
|-------------|-----------------------------------|----------------------------------|
| 入り拍前        | beatNo < startBeatNo              | 演奏せず待機する.                        |
| 休符          | noteNumber == 0                   | NOTE を送らず, 次イベントへ進む.             |
| BEAT 欠損     | beatTimeoutMs 超過                  | 直前 BPM から仮想 BEAT を作り SelfRun する. |
| 遅延 BEAT     | 発音予定時刻を許容値以上過ぎた場合                 | 発音をスキップし, ログに警告を出す.              |
| 同じ BEAT の重複 | seq または beatNo が処理済み              | 二重発音を防ぐため捨てる.                    |
| 曲末          | currentEventIndex >= kScoreLength | デモでは曲頭へ戻る. 提出仕様では停止へ変更可能.        |

| 状況          | 判定条件                                    | 処理                      |
|-------------|-----------------------------------------|-------------------------|
| Serial 送信失敗 | <code>Serial.write()</code> の戻り値が 20 未満 | 送信失敗ログを出し、次 NOTE で復帰する。 |